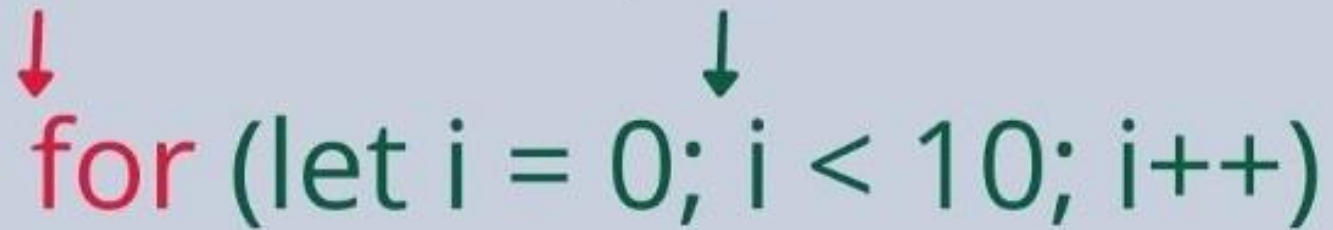


Time Complexity of Loops



```
↓      ↓  
for (let i = 0; i < 10; i++)
```

The efficiency of an algorithm depends on two parameters:

1. Time Complexity
2. Space Complexity

Time Complexity:

Time Complexity is defined as the number of times a particular instruction set is executed rather than the total time taken. It is because the total time taken also depends on some external factors like the compiler used, the processor's speed, etc.

Space Complexity:

Space Complexity is the total memory space required by the program for its execution.

Types of Notations for Time Complexity

O

Big Oh

Upper Bound

Worst Case

Ω

Omega

Lower Bound

Best Case

Θ

Theta

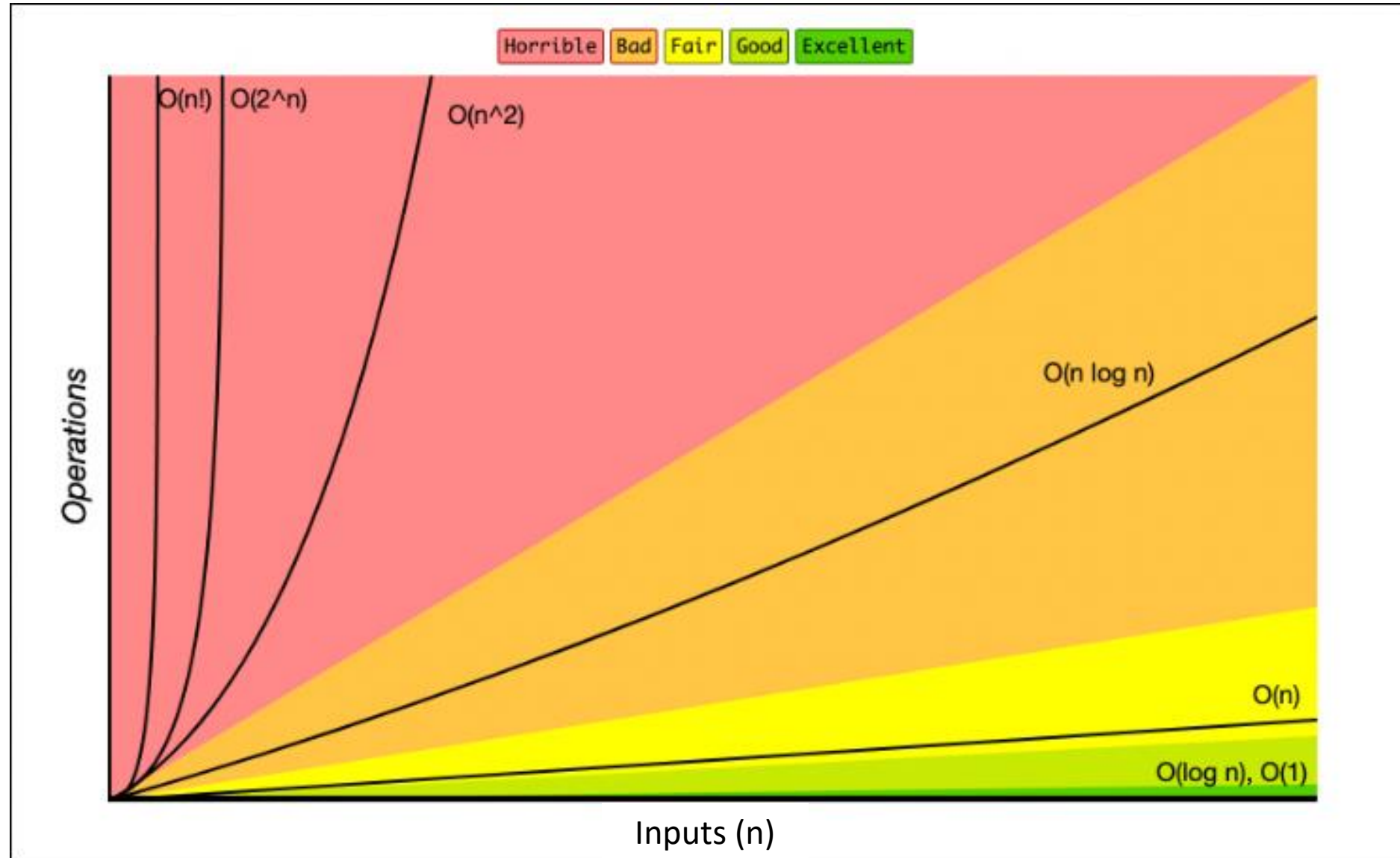
Upper & Lower
Bound

Average 'ish'

Understanding Notations of Time Complexity with Example

- **O(expression)** It indicates the maximum time complexity required by an algorithm for all input values. It represents the worst case of an algorithm's time complexity.
- **Omega(expression)** It indicates the minimum time complexity required by an algorithm for all input values. It represents the best case of an algorithm's time complexity.
- **Theta(expression)** It indicates the average bound of an algorithm. It represents the average case of an algorithm's time complexity.

Big O Complexity Chart



Constant Time: $O(1)$:

When your algorithm is not dependent on the input size n , it is said to have a constant time complexity with order $O(1)$. This means that the run time will always be the same regardless of the input size.

Linear Time: $O(n)$

You get linear time complexity when the running time of an algorithm increases linearly with the size of the input. This means that when a function has an iteration that iterates over an input size of n , it is said to have a time complexity of order $O(n)$.

Logarithm Time: $O(\log n)$

This is similar to linear time complexity, except that the runtime does not depend on the input size but rather on half the input size. When the input size decreases on each iteration or step, an algorithm is said to have logarithmic time complexity.

Quadratic Time: $O(n^2)$

When you perform nested iteration, meaning having a loop in a loop, the time complexity is quadratic, which is horrible.

Increasing Order

General Easy Rules (Remember First)

- **$i++$ or $i += \text{constant}$** $\rightarrow$ linear growth
- **$i += 2$** $\rightarrow$ still linear (half steps)
- **$i * 2$** $\rightarrow$ logarithmic
- **Condition like $i*i < n$** $\rightarrow$ i goes up to $\sqrt{n}$
- **Loop runs n^2 times** $\rightarrow$ quadratic
- **Constants are ignored in Big-O**
- If a variable decreases by a constant amount $\rightarrow O(n)$

Loop 1

```
for (i = 0; i < n; i++)
```

- i starts from 0
- Increases by 1
- Runs n times

Time Complexity: $O(n)$

Loop 2

```
for (i = 0; i < n*n; i++)
```

- Runs until n^2
- $n \times n$ iterations

Time Complexity: $O(n^2)$

Loop 3

```
for (i = 0; i < n; i += 2)
```

- i increases by 2
- $n/2$ iterations

Time Complexity: $O(n)$

Loop 4

```
for (i = 1; i <= n; i *= 2)
```

- i doubles each time
- 1,2,4,8...

Time Complexity: $O(\log n)$

Loop 5

```
for (i = 0; i*i < n; i++)
```

- Condition: $i^2 < n$
- i goes up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Loop 6

```
for (i = 0; i*i < n; i += 2)
```

- Step size 2
- Still up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Loop 7

```
for (i = 1; i*i < n; i *= 2)
```

- i doubles
- Square grows fast

Time Complexity: $O(\log n)$

Loop 8

```
for (i = 1; i <= n; i++)
```

- Simple linear loop
- n iterations

Time Complexity: $O(n)$

Loop 9

```
for (i = 1; i <= n*n; i++)
```

- Runs n^2 times

Time Complexity: $O(n^2)$

Loop 10

```
for (i = 1; i <= n; i += 2)
```

- Only odd numbers
- $n/2$ iterations

Time Complexity: $O(n)$

Loop 11

```
for (i = 0; i < n; i *= 2)
```

- i starts at 0
- $0 \times 2 = 0$

Infinite Loop

Loop 12

```
for (i = 1; i*i <= n; i++)
```

- i up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Loop 13

```
for (i = 1; i*i <= n; i += 2)
```

- Odd values only
- Up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Loop 14

```
for (i = 1; i*i <= n; i *= 2)
```

- i doubles

Time Complexity: $O(\log n)$

Loop 15

```
for (i = 0; i*i < n; i *= 2)
```

- i remains 0

Infinite Loop

Decreasing Order

Easy Rules for Decreasing Loops

- **$i--$ or $i -= \text{constant}$** $\rightarrow$ Linear $\rightarrow O(n)$
- **$i /= 2$** $\rightarrow$ Logarithmic $\rightarrow O(\log n)$
- **Start = n^2** $\rightarrow$ Quadratic $\rightarrow O(n^2)$
- **Condition like $i/i > 0$**
 - $i/i = 1$ for all $i \neq 0$
 - Loop behaves like a normal linear loop
- **Condition $i*i > 0$**
 - True for all $i \neq 0$
 - Loop ends only when $i = 0$

Loop 15

for (i = n; i > 0; i--)

- Starts from n
- Decreases by 1
- Runs n times

Time Complexity: $O(n)$

Loop 16

```
for (i = n*n; i > 0; i--)
```

- Starts from n^2
- Decreases by 1

Time Complexity: $O(n^2)$

Loop 17

```
for (i = n; i > 0; i -= 2)
```

- Decreases by 2
- $n/2$ iterations

Time Complexity: $O(n)$

Loop 18

```
for (i = n; i >= 1; i /= 2)
```

- i is halved each time

Time Complexity: $O(\log n)$

Loop 19

```
for (i = n; i/i > 0; i--)
```

- $i/i = 1$ for $i \neq 0$
- Runs like $i > 0$

Time Complexity: $O(n)$

Loop 20

```
for (i = n; i*i > 0; i -= 2)
```

- $i^2 > 0$ until $i = 0$

Time Complexity: $O(n)$

Loop 21

```
for (i = n; i >= 0; i--)
```

- Runs from n to 0

Time Complexity: $O(n)$

Loop 22

```
for (i = n*n; i >= 0; i--)
```

- Runs from n^2 to 0

Time Complexity: $O(n^2)$

Loop 23

```
for (i = n; i >= 0; i -= 2)
```

- Half steps

Time Complexity: $O(n)$

Loop 24

for (i = n; i >= 0; i /= 2)

- i becomes 0
- $0/2 = 0$

Infinite Loop

Loop 25

```
for (i = n; i >= 1; i /= 2)
```

- Divides until $i = 1$

Time Complexity: $O(\log n)$

Loop 26

```
for (i = n; i/i > 0; i--)
```

- Same as Loop 19

Time Complexity: $O(n)$

Loop 27

```
for (i = n; i*i >= 1; i -= 2)
```

- Runs until $i = 0$

Time Complexity: $O(n)$

Loop 28

```
for (i = n; i*i >= 1; i /= 2)
```

- i halves until < 1

Time Complexity: $O(\log n)$

Loop 29

```
for (i = n; i/i >= 1; i--)
```

- $i/i = 1$
- Runs until $i = 0$

Time Complexity: $O(n)$

Mixed Loops Increasing Order

30 – Sequential Loops

i. for (i = 0; i < n; i++)

→ $O(n)$

ii. for (j = 1; j <= n; j++)

→ $O(n)$

Total Time = $O(n) + O(n)$

Dominant Term = $O(n)$

31 – Sequential Loops

i. for (i = 1; i <= n*n; i++)

→ $O(n^2)$

ii. for (j = 0; j < n; j += 2)

→ $O(n)$

Total Time = $O(n^2) + O(n)$

Dominant Term = $O(n^2)$

32 – Sequential Loops

i. for (i = 0; i < n*n; i++)

→ $O(n^2)$

ii. for (j = 1; j <= n*n; j++)

→ $O(n^2)$

Total Time = $O(n^2) + O(n^2)$

Dominant Term = $O(n^2)$

33 – Sequential Mixed Loops

i. for ($i = 1$; $i * i < n$; $i += 2$)

→ $O(\sqrt{n})$

ii. for ($j = 1$; $j * j < n$; $j *= 2$)

→ $O(\log n)$

Total Time = $O(\sqrt{n}) + O(\log n)$

Dominant Term = $O(\sqrt{n})$

Mixed Loop Decreasing Order

34 – Sequential Loops

i. for ($i = n$; $i > 0$; $i--$)

→ $O(n)$

ii. for ($j = n$; $j > 0$; $j--$)

→ $O(n)$

Total = $O(n) + O(n)$

Final = $O(n)$

35 – Sequential Loops

i. for ($i = n$; $i \geq 0$; $i--$)

→ $O(n)$

ii. for ($j = n * n$; $j > 0$; $j--$)

→ $O(n^2)$

Total = $O(n) + O(n^2)$

Final = $O(n^2)$

36 – Sequential Mixed Loops

i. for ($i = n$; $i \geq 0$; $i -= 2$)

→ $O(n)$

ii. for ($j = n$; $j \geq 1$; $j /= 2$)

→ $O(\log n)$

Total = $O(n) + O(\log n)$

Final = $O(n)$

37 – Sequential Mixed Loops

i. for ($i = n$; $i \geq 1$; $i /= 2$)

→ $O(\log n)$

ii. for ($j = n$; $j/j > 0$; $j--$)

→ $O(n)$

Total = $O(\log n) + O(n)$

Final = $O(n)$

38 – Sequential Mixed Loops

i. for ($i = n$; $i \geq 0$; $i -= 2$)

→ $O(n)$

ii. for ($j = n$; $j \geq 1$; $j /= 2$)

→ $O(\log n)$

Total = $O(n) + O(\log n)$

Final = $O(n)$

NESTED LOOPS

INDEPENDENT + INCREASING ORDER

39 – Nested Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

40 – Nested Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j += 2)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

41 – Nested Loop

```
for (int i = 0; i < n; i += 2)  
    for (int j = 0; j < n; j++)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

42 – Nested Mixed Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 1; j < n; j *= 2)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(\log n)$
- Total = $O(n) \times O(\log n)$
- Final = $O(n \log n)$

NESTED LOOPS

INDEPENDENT + DECREASING ORDER

43 – Nested Loop

```
for (int i = n; i > 0; i--)  
    for (int j = n; j > 0; j--)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

44 – Nested Loop

```
for (int i = n; i > 0; i--)  
    for (int j = n; j > 0; j -= 2)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

45 – Nested Loop

```
for (int i = n; i > 0; i -= 2)
    for (int j = n; j > 0; j--)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(n)$
- Total = $O(n) \times O(n)$
- Final = $O(n^2)$

46 – Nested Mixed Loop

```
for (int i = n; i > 0; i--)  
    for (int j = n; j > 0; j /= 2)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(\log n)$
- Total = $O(n) \times O(\log n)$
- Final = $O(n \log n)$

NESTED LOOPS

DEPENDENT + INCREASING ORDER

47 – Nested Dependent Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j <= i; j++)
```

- Outer Loop $\rightarrow$ n times
- Inner Loop $\rightarrow$ $i + 1$ times
- Total Work = $1 + 2 + \dots + n$
- Final = $O(n^2)$

48 – Nested Dependent Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j <= i; j += 2)
```

- Outer Loop $\rightarrow$ n times
- Inner Loop $\rightarrow$ $i/2$ times
- Total Work $\approx n^2$
- Final = $O(n^2)$

49 – Nested Mixed Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 1; j < i; j *= 2)
```

- Outer Loop $\rightarrow O(n)$
- Inner Loop $\rightarrow O(\log i)$
- Total $\approx n \log n$
- Final = $O(n \log n)$

50 – Nested Square-Root Loop

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j*j <= i; j++)
```

- Inner Loop $\rightarrow \sqrt{i}$ times
- Total Work $\approx n^{3/2}$
- Final = $O(n^{3/2})$

Arrangement of Time Complexity

Most Efficient → Least Efficient

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^2 \log n) < O(n^3) < O(2^n) < O(3^n) < O(n!)$$

What This Arrangement Means

- **Left side** algorithms grow **slowly** → more efficient
- **Right side** algorithms grow **very fast** → less efficient
- Exponential and factorial algorithms become impractical very quickly

IDENTIFICATION RULES

Code Pattern	Complexity
No loop	$O(1)$
One loop	$O(n)$
Nested loops	$O(n^2)$
Divide by 2 ($i = i / 2$)	$O(\log n)$
Squaring variable ($i = i * i$)	$O(\log \log n)$
Square root loop ($i * i \leq n$)	$O(\sqrt{n})$
Recursion (2 calls)	$O(2^n)$
All permutations	$O(n!)$